REPORT

Numbering Systems

Full Name	
Student ID	
Submission Date	
GitHub Repo	

Academic Integrity Declaration

I declare that this report is my own original work. All code was written by me. I have not shared my answers, code, or output with any other student.

```
Name       : _____________________________________
Student ID : _______________
Date       : _______________
```

Report Overview

This report documents your findings for the Assignment of Numbering Systems. Each section contains structured answer boxes and fill-in tables. For every code output box, paste the exact screenshot from running your Python script. For every written answer box, write your explanation in your own words.

Task A — Number Conversion Drill

For each problem: (1) complete the working steps table by hand, (2) paste the matching output from task_a.py in the code output box.

Q1 — Convert 0x4A to Decimal

Method: Positional sum 0x4A → decimal *[positional sum]*

Power of 16	Hex digit	Digit value	× Power	Subtotal
Result:				

Code output — task_a.py Q1 result:

Q2 — Convert 0x4A to Binary

Use the nibble method: split 0x4A into two nibbles (4 and A), look up each in the nibble table.

Method: Nibble grouping 0x4A → binary *[nibble method]*

Hex digit	Nibble (4-bit binary)	Working
Result:		

Code output — task_a.py Q2 result:

Q3 — Convert 42 Decimal to Binary

Method: Repeated division by 2 42_{10} → binary *[repeated ÷2]*

Dividend	÷ 2	Quotient	Remainder
Result:			

Read remainders: bottom → top → result: _____

Code output — task_a.py Q3 result:

Q4 — Convert 11010110_2 to Hexadecimal

Method: Nibble grouping (4-bit groups from right) 11010110 → hex *[nibble method]*

Bits (left group)	Bits (right group)	Left hex digit	Right hex digit
Result:			

Code output — task_a.py Q4 result:

Q5 — Convert 11010110_2 to Decimal

Method: Positional sum 11010110 → decimal *[positional sum]*

Bit position	Bit value	Place value (2^n)	Contribution
Result:			

Code output — task_a.py Q5 result:

Q6 — Convert 0xFF: Unsigned vs Signed int8

Unsigned int8:	**Signed int8 (two's complement):**
`0xFF = _____ decimal`	`0xFF = _____ decimal`
Range: 0 to 255	*Range: -128 to +127 | If > 127: subtract 256*

Explanation — why the same bit pattern 0xFF gives two different values:

Code output — task_a.py Q6 result:

Q7 — chmod 755 in Binary

Owner (u)	Group (g)	Other (o)
Octal: 7	Octal: 5	Octal: 5
`Binary: ___`	`Binary: ___`	`Binary: ___`
Meaning: rwx	*Meaning: r-x*	*Meaning: r-x*

Code output — task_a.py Q7 result:

Q8 — Port 443 to Hexadecimal

Method: Repeated division by 16
443_{10} → hex *[repeated ÷16]*

Dividend	÷ 16	Quotient	Remainder	Hex digit
Result:				

Code output — task_a.py Q8 result:

Q9 — IPv4 10.0.0.1 to Hex

Octet 10	Octet 0	Octet 0	Octet 1
10 decimal	0 decimal	0 decimal	1 decimal

0x ___	0x ___	0x ___	0x ___

Complete hex: 10.0.0.1 = __ __ __ __

Code output — task_a.py Q9 result:

Q10 — Little-Endian Byte Order Correction

As seen in hex dump (left → right):	**After little-endian correction:**
78 56 34 12 [Address 0x10] [0x11] [0x12] [0x13]	____ ____ ____ ____ → 0x_____ = _____ decimal

Explanation — why do x86-64 systems store values in little-endian order?

Code output — task_a.py Q10 result:

Task A — Script Summary Output

Paste the complete terminal output from running: python3 task_a.py below.

Complete output of task_a.py (all 10 questions):

Task B — File Investigation

B1 — Magic Bytes: File Type Identification

Question	Your answer
Four bytes at offset 0x00 (hex notation):	*FF __ __ __*
File format identified:	
Which bytes triggered identification:	
Why are they called 'magic bytes':	

Code output — task_b.py B1 (read_magic_bytes):

B2 — Unix Timestamp at Offset 0x10

Step	Your working / answer
Step 1 — Raw bytes at 0x10:	__ __ __ __
Step 2 — After little-endian reversal (hex):	__ __ __ __ → 0x_____
Step 3 — Decimal value:	
Step 4 — Human-readable UTC date and time:	*YYYY-MM-DD HH:MM:SS UTC*

Code output — task_b.py B2 (read_unix_timestamp):

B3 — UTF-16LE String at Offset 0x20

Character decode table — fill in each 2-byte pair and its ASCII character:

Offset	Byte 1 (char byte)	Byte 2 (padding)	Decimal value	ASCII character
0x20				
0x22				
0x24				
0x26				
0x28				
0x2A (null)	00	00	— *null terminator*	*end of string*

Decoded string: "_____"

Code output — task_b.py B3 (read_utf16le_string):

B4 — 32-bit Integer at Offset 0x30

Step	Your working / answer
Step 1 — Raw bytes at 0x30	__ __ __ __
Step 2 — After little-endian reversal:	__ __ __ __
Step 3 — Corrected hex value:	*0x*_____
Step 4 — Decimal value:	
Step 5 — Identified network service (if any):	*Port* ____ = _____

Code output — task_b.py B4 (read_le_int32):

Task B — Complete Script Output

Paste the full terminal output from running: python3 task_b.py below.

Complete output of task_b.py:

Task C — XOR Obfuscation Analysis

C1 — Extract the XOR Key

Question	Answer
XOR key — hex value:	*0x __*
XOR key — decimal value:	
Prove self-inverse: key XOR key =	*0x__ XOR 0x__ = ____*
What does the result prove?	

Explain the 'pattern visibility' property: how could an investigator find a single-byte XOR key from a hex dump without knowing the original plaintext? (Hint: what happens to null bytes?)

C2 — Manual XOR Decode Table

Decode the first 8 payload bytes (offsets 0x01 to 0x08) step by step. Fill in every cell. Show the XOR working for each byte.

Offset	Encoded byte (hex)	XOR key (hex)	Decoded byte (hex)	Binary working (optional)
0x01				
0x02				
0x03				
0x04				
0x05				
0x06				
0x07				

0x08				

First 4 decoded bytes (hex):	What they spell in ASCII:
__ __ __ __	" . __ __ __ __ "

What does the decoded ASCII string in the first 4 bytes tell you about the hidden file type?

C3 — Identify the Hidden File Type & Forensic Analysis

Question	Answer
Magic bytes (first 4 decoded bytes in hex):	__ __ __ __
File format identified (from magic bytes):	
File format confirmed by Python script:	

Forensic interpretation: why would an attacker XOR-encode a file of this specific type? What are your next investigative steps after identifying it?

Task C — Script Output

Complete output of task_c.py: